

Chapter 7 pt 1: Reading and Writing Files

EEC 3150

Trevecca Nazarene University

Outline

- 1 Basic of Reading Files
- 2 Extracting Data from Files
- 3 Writing to Files
- 4 Formatting Strings
- 5 Formatting Strings

Basic of Reading Files

Basic of Reading Files

Intro to reading a file with Python:

```
fname = "FirstFile.txt"

fhandle = open(fname, 'r')

string = fhandle.read()
print(string)

fhandle.close()
```

- To read a file, use the `open()` function
- Two string arguments: 1) the filename, 2) whether to read ('r'), write ('w'), or append ('a') to the file
- Returns a *file handle* (essentially a variable that points to the file)
- Since file handles are objects, they have their own methods (such as `read()`)
- Close the file when finished
- Trying to read after closing throws an error

Basic of Reading Files

Reading files in a with,as block:

- Closing a file is annoying and you can forget to do it
- For convenience, we can read files inside a with,as block

```
fname = "FirstFile.txt"
with open(fname, 'r') as fhandle:
    string = fhandle.read()
    print(string)
```

- The file handle is only defined in the scope of the block
- Variables you define inside the block will still be defined after the block
- The file is closed automatically after the block is finished
- Read the file in the block, save the contents to a variable, and use them later on

Basic of Reading Files

Some read methods:

```
with open(fname, 'r') as fhandle:  
    string = fhandle.read()
```

`fhandle.read()`

- Returns the entire file in one string

```
with open(fname, 'r') as fhandle:  
    lines = fhandle.readlines()
```

`fhandle.readlines()`

- Returns a list of strings split on newlines

Basic of Reading Files

Reading a line at a time:

```
with open(fname, 'r') as fhandle:  
    for line in fhandle:  
        print(line)
```

```
with open(fname, 'r') as fhandle:  
    line = fhandle.readline()  
    print(line)  
    line = fhandle.readline()  
    print(line)
```

With a for loop:

- We can read a line at a time with a for loop

`fhandle.readline()`

- Returns the next line

Basic of Reading Files

Reading a line at a time:

```
with open(fname, 'r') as fhandle:
    line = fhandle.readline()
    while line:
        print(line)
        line = fhandle.readline()
```

With a while loop:

- We can read a line at a time with a while loop
- Read the first line
- Use the line as the loop condition
- Once `readline()` returns an empty string, the loop exits

Basic of Reading Files

Filepaths:

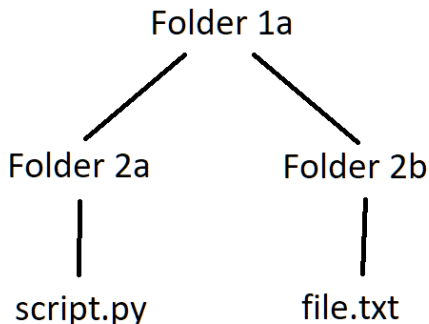
- Unless specified, Python assumes that the file you want to read/write is in the same folder/directory as the script
- If it isn't, you must provide a *filepath* for the file (its location on your computer's hard drive)

Two kinds of filepaths (say our script is in the Python directory):

- **Absolute:** a filepath from the root directory of the drive to the file:
Ex: C:/Users/Graham West/Python/Data/FirstFile.txt
- **Relative:** a filepath from the script directory to the file:
Ex: Data/FirstFile.txt

Basic of Reading Files

Parent directory command: `..`



- If you need to navigate to the parent directory, you can include `“..”` in your relative filepath:
Ex: `../Folder 2b/file.txt`

Extracting Data from Files

Extracting Data from Files

Get a list of grades from a file:

Read the file Grades.txt and produce a list of integer grades.

- This file is simple and has one number per line

Get lists of years and temperatures from a file:

Read the file YearTemp.txt and produce lists of integer years and float temperatures.

- This file is more complicated
- It has a header that we must ignore
- Year and temp values are comma-separated (one pair per line)

Extracting Data from Files

Get 2D list galaxy simulation parameters:

Read the file `587722984435351614_combined.txt` and produce a 2D list of simulation parameters and a 1D list of simulation fitnesses.

- File has no header
- Each line has two comma-separated sections which have a tab n/t them
- At a certain line number, there are no more fitness values, so stop reading here

Extracting Data from Files

Get electron density parameters from files:

Write a program that can read any of the following files (`h2o.txt`, `nh3.txt`, `ch4.txt`) and create lists of the CENTRE ASSIGNMENTS (note the spelling), TYPE ASSIGNMENTS, and EXPONENTS.

- The values span multiple lines
- Each file spans a different number of lines
- The program should be able to handle all possible cases

Writing to Files

Writing to Files

Writing files in a with,as block:

```
fname = "Write.txt"
with open(fname, 'w') as f:
    f.write('test')
```

- Open the file in the same as before, but now with 'w'
- Use the write() function

The writelines() function:

```
fname = "Write.txt"
with open(fname, 'w') as f:
    x = ['1\n', '2\n']
    f.writelines(x)
```

- The writelines() function can accept a list of strings and write all of them to a file
- HOWEVER, despite its name, it doesn't automatically add newlines (\n) at the end of each string in the list; you must do it manually

Writing to Files

Write a csv file:

Write a program that will write a 2D list of numbers to a file. The numbers on each line should be comma-separated.

Formatting Strings

Formatting Strings

The `str.format()` method:

The `str.format()` method lets you manually format string in various ways:

```
"My name is {fname}, I'm {age}".format(name="John",age=36)
```

```
"My name is {0}, I'm {1}".format("John",36)
```

```
"My name is {}, I'm {}".format("John",36)
```

```
My name is John, I'm 36
```

- Call the method with the string you want to format
- Insert args of function into the string with curly brackets: `{}`
- The args can be inserted via keywords, numbers, or in order of appearance (by default, last example above)

Formatting Strings

Padding strings:

The real power of `str.format()` lies in *padding* (insertion of whitespace for readability):

- `‘‘{:10}’’.format(‘‘test’’)`
This will return string that is 10 characters long and starts with “test” (extra characters are spaces)
- `‘‘{:>10}’’.format(‘‘test’’)`
This will return string that is 10 characters long and ends with “test” (extra characters are spaces)

Why do we care?

- If we want to print/write multiples of different length, we can pad them so that they are aligned

Padding with floats:

Padding is very useful when dealing with floating point numbers with different numbers of decimals:

- `‘‘{:m.nf}’’ .format(number)`
m: the total number of characters (the length)
n: the number of decimal places (padded with zeroes)
- `‘‘{:10.4f}’’ .format(100.01)`
- To assign these numbers from variables, simply add more curly brace pairs:
`‘‘{:}.{}.f’’ .format(100.01,10,4)`
- If this is confusing, add the argument indices:
`‘‘{0:}{1}.{2}f’’ .format(100.01,10,4)`

Write formatted numbers to a file:

Write a program that will

- write a list of floats to a file
- in a right justified fashion
- with all decimals aligned
- with a consistent number of decimals
- by passing the length and number of decimals as an argument to `format()`
- and determine the minimum value of these numbers from the list

Appending to Files

Opening a file in append mode: 'a'

```
fname = "Write.txt"
with open(fname, 'a') as f:
    x = ['1\n', '2\n']
    f.writelines(x)
```

- To add lines to an existing file without overwriting it, open it in append mode using 'a'